DATA PRODUCT CASE STUDY · STELLARALGO

One source of truth for a multi-tenant product

How I turned siloed client data into one governed layer for reporting, AI and the tools teams work in.

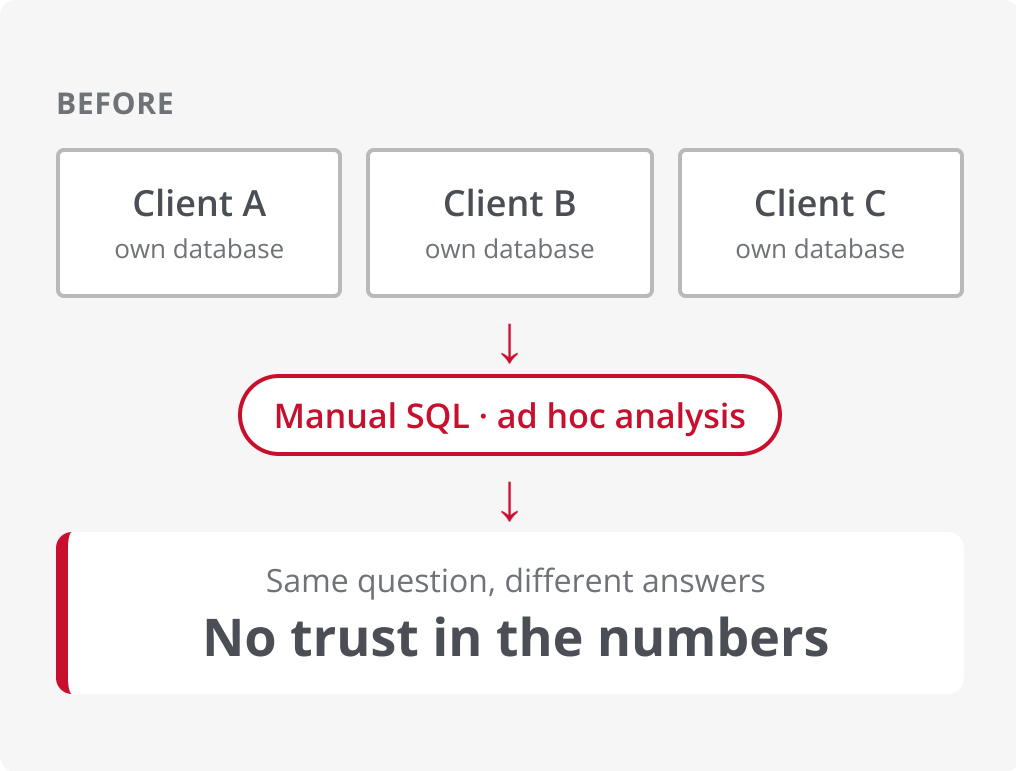
Cody Chandler
Data and Business Analyst interview · The City of Calgary

I'll answer them in order

1. What business problem was I solving?
2. What is the data product?
3. How did I approach it?
4. What challenges came up?
5. What am I most proud of?
6. What would I improve with more time?

What business problem was I solving?

The problem to solve: no single analytical source of truth, so reporting, analysis and AI couldn't be consistent or reliable at scale.



! What was happening

Each client's data lived in its own database. Reports were up to **7 days old**, and nobody owned the numbers.

a Why it mattered

Every cross-client question meant manual SQL and ad hoc analysis, and teams still ended up with different numbers.

b Users and interested parties

Customer Success · Product · Clients · Analysts

What is the data product?

a One governed, tested layer for every client's data, refreshed four times a day.



GOVERNED THROUGHOUT · Role-based access · Data contract · Freshness monitoring · Tested every run

Transformed once, in one place, then reused by every team and tool.

What is the data product?

b How people use it: one layer, three ways in.

BI dashboards

Self-serve reporting on the tested gold tables.

Leadership, Product, Customer Success

AI assistant

Ask in plain English. The semantic layer picks the model with the right grain.

“How did revenue trend this month?”

Where they work

Gold data and ML results pushed into the CRM, right where account teams already work.

CS and Account Management

Every channel reads the same governed layer, so every number matches.

What is the data product?

c The value it delivered.

FRESH

~~Weekly, by hand~~

4× a day

Automated refreshes, on a schedule teams can rely on.

CONSISTENT

~~Different answers~~

One number

The same metric in every report, dashboard and tool.

OBSERVABLE

~~Found by clients~~

Monitored

Freshness, tests, pipeline errors and cost, watched on every run.

SELF-SERVE

~~Engineering tickets~~

Self-serve

Routine Customer Success questions, answered without engineering.

Every team, dashboard and tool now works from one consistent, reliable source of truth.

How did I approach it?

Start from what people need, fix the data once, then keep improving it.

1 Requirements
The questions each team couldn't answer, and the numbers they disputed.

2 Strategy
One governed layer, fixed once instead of report by report.

3 Plan
Medallion layers, a data contract, one grain per table.

4 Build
dbt models in Snowflake, orchestrated by Airflow.

5 Review & iterate
Tests and monitoring find issues; fix them once, in the layer.

a Sourcing and preparing the data

Product analytics, customer data and client config.

Land
Raw, personal data masked

→

Map
Client by client, into one model

→

Test
On every run, before use

b Methods and tools

BUILT IN **dbt**	STORED IN **Snowflake**	ORCHESTRATED BY **Airflow on AWS**
VERSIONED IN **GitHub**	DOCUMENTED IN **dbt docs, Confluence**	SERVED THROUGH **Semantic layer, QuickSight, Claude**

What challenges came up, and how did I handle them?

Three kinds of challenge, each fixed in the layer rather than in reports.

Type	Challenge	How I solved it
Technical	Every client system had its own shape.	One model all clients map into.
Analytical	The same metric meant different things to different teams.	One definition, one documented grain per table.
People	Nobody owned freshness or accuracy.	A data contract I wrote: owners, targets and incident tiers.

Every fix lived in the layer, so it fixed every report at once.

What am I most proud of?

AI answers that match the dashboards, by design.

QUESTION

"How did ticket revenue trend this month?"

→

SEMANTIC LAYER: ROUTE TO THE RIGHT GRAIN

Property by day	One row per property, per day ✓
Campaign	One row per campaign
Other models	Each with one documented grain

→

ANSWER

Same number as the dashboard

SQL shown, within the user's role

Governance stopped being overhead and became the reason people could self-serve.

What would I improve with more time?

Prove the value sooner, and take the layer further.



Measure from day one

Baseline hours and escalations before launch, so the value is a number.



Take it to clients

Offer the internal AI assistant to clients, on the same layer.



Cover more domains

More semantic models, each with one grain, so the AI can answer more.

The same work at the City: find where numbers disagree, agree one definition with the owners, and make it self-serve.

Thank you.

I can't show you StellarAlgo's data, so I rebuilt the same approach on City of Calgary open data. Let's look at it live.

LIVE DEMO

codychandler.ca/data-demos/calgary-transit